

# EduFrame Subsystems & Design Pattern Specification

## Overview

This document specifies the software design pattern assigned to each of the 6 main EduFrame system components based on the lecture series (**Design Patterns Part 01 & Part 02**) and the **EduFrame System Architecture Design Document**.

## Component & Design Pattern Mapping Table

| Subsystem | Component Name                 | Group Member & IT Number            | Design Pattern                  | Pattern Category |
|-----------|--------------------------------|-------------------------------------|---------------------------------|------------------|
| UC-01     | Announcement & Event System    | De Silva L. C. A. (IT25101841)      | Observer Pattern                | Behavioral       |
| UC-02     | Support Ticketing System       | Munasinghe M. H. V. S. (IT25102695) | Chain of Responsibility Pattern | Behavioral       |
| UC-03     | Advertisement Management       | Bandara W. G. M. D. (IT25103702)    | Singleton Pattern               | Creational       |
| UC-04     | Course & Curriculum Management | Withana T. T. (IT25101767)          | Composite Pattern               | Structural       |
| UC-05     | Video Management System        | Wickramathunga P. P. (IT25103673)   | Strategy Pattern                | Behavioral       |
| UC-06     | Quiz & Assessment Subsystem    | Adikari A. M. T. D. (IT25100859)    | Factory Method Pattern          | Creational       |

## Subsystem Details & Concrete Code Examples

### 1. Announcement & Event System (UC-01)

- **Assigned Member:** De Silva L. C. A. (IT25101841)
- **Design Pattern:** Observer Pattern (Behavioral)
- **Description:** Whenever a teacher creates or updates a time-sensitive course event or announcement, the system notifies all registered/subscribed student observers automatically.

```
// Subject / Publisher Contract
public interface AnnouncementSubject {
    void registerObserver(NotificationObserver observer);
    void removeObserver(NotificationObserver observer);
    void notifyObservers(AnnouncementDTO announcement);
}

// Concrete Subject (Spring Event Publisher / Service)
@Service
public class AnnouncementService implements AnnouncementSubject {
    private final List<NotificationObserver> observers = new ArrayList<>();
}
```

```

@Override
public void registerObserver(NotificationObserver observer) {
    observers.add(observer);
}

@Override
public void notifyObservers(AnnouncementDTO announcement) {
    for (NotificationObserver observer : observers) {
        observer.update(announcement);
    }
}

public Announcement createAnnouncement(AnnouncementDTO dto) {
    Announcement saved = announcementRepository.save(new Announcement(dto));
    notifyObservers(dto); // Automatically notifies all subscribed students
    return saved;
}
}

// Concrete Observer (Student Dashboard Notification Listener)
@Component
public class StudentDashboardNotificationListener implements NotificationObserver {
    @Override
    public void update(AnnouncementDTO announcement) {
        System.out.println("Push notification dispatched to Student Feed: " + announcement.getTitle(
));
    }
}
}

```

## 2. Support Ticketing System (UC-02)

- **Assigned Member:** Munasinghe M. H. V. S. (IT25102695)
- **Design Pattern:** Chain of Responsibility Pattern (Behavioral)
- **Description:** Incoming technical support tickets are passed sequentially down a handler chain (AutoBotHandler -> StaffHandler -> AdminHandler) until a handler resolves the issue or escalates it.

```

// Abstract Handler in Chain
public abstract class TicketHandler {
    protected TicketHandler nextHandler;

    public void setNextHandler(TicketHandler nextHandler) {
        this.nextHandler = nextHandler;
    }

    public abstract void processTicket(TicketDTO ticket);
}

// Handler 1: Automated Bot Handler for Simple Queries
public class AutoBotHandler extends TicketHandler {
    @Override
    public void processTicket(TicketDTO ticket) {
        if ("PASSWORD_RESET".equalsIgnoreCase(ticket.getIssueType())) {
            ticket.setResolution("Automated reset link dispatched to " + ticket.getUserEmail());
            ticket.setStatus("RESOLVED");
        } else if (nextHandler != null) {
            nextHandler.processTicket(ticket); // Delegate down the chain
        }
    }
}

// Handler 2: Staff Support Handler for Standard Issues
public class SupportStaffHandler extends TicketHandler {
    @Override

```

```

    public void processTicket(TicketDTO ticket) {
        if ("VIDEO_BUFFERING".equalsIgnoreCase(ticket.getIssueType())) {
            ticket.setResolution("Staff reviewed stream bitrate and re-indexed module.");
            ticket.setStatus("RESOLVED");
        } else if (nextHandler != null) {
            nextHandler.processTicket(ticket); // Escalate to Admin
        }
    }
}

```

### 3. Advertisement Management System (UC-03)

- **Assigned Member:** Bandara W. G. M. D. (IT25103702)
- **Design Pattern:** Singleton Pattern (Creational)
- **Description:** Ensures a single global manager instance (AdvertisementManager) controls promotional banner allocation, pre-roll/mid-roll playback scheduling, and active expiry tracking system-wide.

```

public class AdvertisementManager {
    // Single thread-safe static instance
    private static volatile AdvertisementManager instance;
    private final List<Advertisement> activeCampaigns;

    private AdvertisementManager() {
        this.activeCampaigns = new CopyOnWriteArrayList<>();
    }

    public static AdvertisementManager getInstance() {
        if (instance == null) {
            synchronized (AdvertisementManager.class) {
                if (instance == null) {
                    instance = new AdvertisementManager();
                }
            }
        }
        return instance;
    }

    public List<Advertisement> getActivePreRollAds() {
        LocalDate today = LocalDate.now();
        return activeCampaigns.stream()
            .filter(ad -> "ACTIVE".equals(ad.getStatus()) && !ad.getEndDate().isBefore(today))
            .collect(Collectors.toList());
    }
}

```

### 4. Course & Curriculum Management System (UC-04)

- **Assigned Member:** Withana T. T. (IT25101767)
- **Design Pattern:** Composite Pattern (Structural)
- **Description:** Treats individual resources (video lectures, downloadable PDFs) and container groups (modules, sub-modules) uniformly under a single composite interface.

```

// Component Interface
public interface CurriculumComponent {
    String getTitle();
    int getDurationMinutes();
    void renderSyllabusView();
}

// Leaf Node: Individual Video Lesson

```

```

public class VideoLessonLeaf implements CurriculumComponent {
    private String title;
    private int durationMinutes;

    public VideoLessonLeaf(String title, int durationMinutes) {
        this.title = title;
        this.durationMinutes = durationMinutes;
    }

    @Override
    public int getDurationMinutes() { return durationMinutes; }

    @Override
    public void renderSyllabusView() {
        System.out.println(" - Video: " + title + " (" + durationMinutes + " min)");
    }
}

// Composite Node: Course Module containing multiple Lessons/Sub-modules
public class CourseModuleComposite implements CurriculumComponent {
    private String moduleTitle;
    private List<CurriculumComponent> children = new ArrayList<>();

    public CourseModuleComposite(String moduleTitle) {
        this.moduleTitle = moduleTitle;
    }

    public void addComponent(CurriculumComponent component) {
        children.add(component);
    }

    @Override
    public int getDurationMinutes() {
        return children.stream().mapToInt(CurriculumComponent::getDurationMinutes).sum();
    }

    @Override
    public void renderSyllabusView() {
        System.out.println("Module: " + moduleTitle + " [Total: " + getDurationMinutes() + " min]");
        for (CurriculumComponent child : children) {
            child.renderSyllabusView();
        }
    }
}

```

## 5. Video Management System (UC-05)

- **Assigned Member:** Wickramathunga P. P. (IT25103673)
- **Design Pattern:** Strategy Pattern (Behavioral)
- **Description:** Encapsulates media file upload, format verification, and storage processing (VideoUploadStrategy, ImageUploadStrategy) into interchangeable strategy implementations.

```

// Common Strategy Interface
public interface MediaUploadStrategy {
    boolean supports(MultipartFile file);
    void handleMedia(Advertisement advertisement, String fileUrl);
}

// Concrete Strategy 1: Video Media Strategy
@Component
public class VideoUploadStrategy implements MediaUploadStrategy {
    @Override
    public boolean supports(MultipartFile file) {
        String contentType = file.getContentType();
    }
}

```

```

        return contentType != null && contentType.startsWith("video/");
    }

    @Override
    public void handleMedia(Advertisement advertisement, String fileUrl) {
        advertisement.setMediaType("VIDEO");
        advertisement.setVideoUrl(fileUrl);
    }
}

// Concrete Strategy 2: Image Media Strategy
@Component
public class ImageUploadStrategy implements MediaUploadStrategy {
    @Override
    public boolean supports(MultipartFile file) {
        String contentType = file.getContentType();
        return contentType != null && contentType.startsWith("image/");
    }

    @Override
    public void handleMedia(Advertisement advertisement, String fileUrl) {
        advertisement.setMediaType("IMAGE");
        advertisement.setImageUrl(fileUrl);
    }
}

```

## 6. Quiz & Assessment Subsystem (UC-06)

- **Assigned Member:** Adikari A. M. T. D. (IT25100859)
- **Design Pattern: Factory Method Pattern** (Creational)
- **Description:** Dynamically instantiates the appropriate evaluation/grading handler (McqGradingStrategy, TrueFalseGradingStrategy, ShortAnswerGradingStrategy) based on the submitted question type.

```

// Factory Class
@Component
public class GradingStrategyFactory {
    private final Map<QuestionType, GradingStrategy> strategyMap;

    public GradingStrategyFactory(List<GradingStrategy> strategies) {
        strategyMap = new EnumMap<>(QuestionType.class);
        for (GradingStrategy strategy : strategies) {
            strategyMap.put(strategy.getSupportedType(), strategy);
        }
    }

    public GradingStrategy getStrategy(QuestionType type) {
        GradingStrategy strategy = strategyMap.get(type);
        if (strategy == null) {
            throw new IllegalArgumentException("No grading strategy registered for type: " + type);
        }
        return strategy;
    }
}

// Usage in QuizAttemptService
GradingStrategy strategy = gradingStrategyFactory.getStrategy(question.getQuestionType());
GradingResult result = strategy.grade(question, studentSubmittedAnswer);

```